

LCNRM — Liability Condition Requirement Module

LCNRM — Liability Condition Requirement Module

Architecture: LIGA™ — Liability Governance Architecture
Parent Standard: Liability Conditions Standard (LCOS)
Category: Governance & Enforcement
Subcategory: Liability Governance
Type: Core Module
Governed Space: Liability Condition Requirements
Module ID: LIGA-LCOS-LCNRM-001
Origin ID: OOF.CM.LIGA.05.02.01
Version: 1.0
Status: Canonical · Open Standard
Effective Date: 22 September 2026
Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem
AI-Readable: Yes
Authority: OOF® — OriginOpen® Foundation
Protection: MIP® — Methodological Intellectual Property
Canonical Language: English (UCL™)

Canonical Definition

The Liability Condition Requirement Module (LCNRM) governs how each Identified Liability Condition is translated into an explicit, bounded, traceable, and evaluable requirement defining what must be true, false, present, absent, occur, not occur, reach a threshold, exist for a defined period, or otherwise be established for that Condition to be evaluated under the applicable Governed Liability Basis.

LCNRM converts Condition identity into Condition evaluation logic without determining the resulting Condition State.

It governs the transition:

Identified Liability Condition Set → Requirement Formation → Requirement Structure → Applicability Parameters → Evaluation Criteria → Governed Liability Condition Requirement Set Its foundational distinction is:

Condition Requirement ≠ Condition Satisfaction.

Operational Role

The operational role of LCNRM is:

to define exactly what each identified Liability Condition requires before its state can be evaluated.

LCNIM answers:

Which Conditions require governance?

LCNRM answers:

What exactly must each Condition require?

The next module determines whether those Requirements are actually satisfied.

Position in the Parent Standard Flow

The internal LCOS progression is:

Liability Condition Identification → Liability Condition Requirement → Liability Condition State → Liability Condition Exception → Liability Condition Continuity LCNRM governs:

Liability Condition Requirement

Its principal input is:

Identified Liability Condition Set

Its principal output is:

Governed Liability Condition Requirement Set

Its canonical handoff is:

Governed Liability Condition Requirement Set → Liability Condition State Governance

Central Operational Question

What exactly must each identified Liability Condition require so that its state can be evaluated consistently and without ambiguity?

Governed Space

LCNRM governs:

Condition Requirement Formation, Requirement Identity, Requirement Source, Requirement Meaning, Requirement Scope, Requirement Subject, Requirement Relationship, Requirement Object, Requirement Basis, Required State, Required Presence, Required Absence, Required Action, Required Non-Action, Threshold Requirements, Temporal Requirements, Alternative Requirements, Concurrent Requirements, Cumulative Requirements, Sequential Requirements, Dependent Requirements, Conditional Requirements, Requirement Parameters, Evaluation Criteria, Requirement Uncertainty, Requirement Conflict, and Requirement Sufficiency.

Condition Requirement

A Liability Condition Requirement defines the state or criterion against which an identified Condition must later be evaluated.

A Requirement may specify that something must:

exist, not exist, occur, not occur, be present, be absent, reach a threshold, remain below a threshold, occur within a defined time, persist for a defined duration, precede or follow another state, or satisfy another explicitly governed criterion.

The Requirement defines the test.

It does not provide the result.

Condition and Requirement

An Identified Liability Condition establishes what conditional matter requires governance.

The Requirement establishes what must be evaluated about that Condition.

Therefore:

Condition Identity ≠ Condition Requirement.

A single Condition may contain several Requirement components where necessary.

Requirement and Condition State

LCNRM defines the Requirement.

LCNSM later determines the Condition State against that Requirement.

Therefore:

Condition Requirement ≠ Condition State.

and:

Condition Requirement ≠ Condition Satisfaction.

Requirement Identity

Each materially distinct Requirement should possess a stable governed reference.

Requirement Identity should preserve its association with:

Condition Identity, Liability Basis, relevant Subject, Relationship, Object, and applicable temporal context.

This allows later State governance to identify exactly which Requirement was evaluated.

Requirement Source

Every Requirement should remain traceable to the governed source that makes it relevant.

The source may include:

the Governed Liability Basis, an obligation referenced by the Basis, a governed rule, a Relationship, an authority structure, a threshold structure, or another applicable governed input.

Therefore:

Requirement Source $\neq$ Requirement Satisfaction.

Requirement Meaning

A Requirement must be expressed clearly enough that two implementations evaluating the same governed inputs can identify what is being tested.

Ambiguous formulations should be decomposed or clarified.

For example:

“adequate control” is not sufficiently evaluable unless the applicable governance structure establishes what qualifies as adequate for the Condition.

Therefore:

Requirement Label $\neq$ Evaluable Requirement.

Requirement Scope

Every Requirement should define its materially relevant scope.

Scope may identify:

the Subject, Relationship, Object, activity, state, period, threshold, or other boundary to which the Requirement applies.

Therefore:

Requirement Scope $\neq$ Liability Scope.

LCNRM scopes the Condition Requirement, not Liability itself.

Subject-Specific Requirement

A Requirement may apply to a specific Liability Subject.

Example structure:

Subject A must possess State X.

The Requirement should identify the relevant Subject explicitly.

Therefore:

Requirement for Subject A $\neq$ Requirement for Subject B by default.

Relationship-Specific Requirement

A Requirement may apply only within a defined Liability Relationship.

Example:

Within Relationship R, Subject A must perform Action X.

The same Subject may therefore face different Requirements in different Relationships.

Therefore:

Same Subject ≠ Same Requirement across all Relationships.

Object-Specific Requirement

A Requirement may concern a specific Governed Liability Object or Object component.

Example:

Object O must possess State X during Period T.

Object boundaries should remain explicit.

Therefore:

Requirement for Object A ≠ Requirement for Object B by default.

Basis-Specific Requirement

Every Requirement should remain associated with the Liability Basis under which it operates.

The same apparent Condition may have different Requirements under different Bases.

Therefore:

Requirement under Basis A ≠ Requirement under Basis B by default.

Positive Requirement

A Positive Requirement specifies that a defined state must:

exist, occur, be present, or otherwise be established.

Canonical structure:

Required State X must exist.

LCNRM defines X.

LCNSM later evaluates whether X existed.

Negative Requirement

A Negative Requirement specifies that a defined state must:

not exist, not occur, be absent, or remain prohibited.

Canonical structure:

Disqualifying State X must not exist.

Therefore:

Negative Requirement ≠ Established Absence.

The Requirement defines required absence; State governance determines whether absence is established.

Action Requirement

A Requirement may specify that a defined action must occur.

It should identify where materially necessary:

who must act, what action is required, in relation to what, and within what relevant boundary.

Therefore:

Required Action ≠ Performed Action.

Non-Action Requirement

A Requirement may specify that a defined action must not occur.

The architecture should distinguish:

required non-action, from:

absence of information about action.

Therefore:

No Evidence of Action $\neq$ Established Required Non-Action.

Threshold Requirement

A Threshold Requirement defines a measurable or otherwise governable boundary.

It should establish:

the governed variable, the threshold, the comparison logic, and relevant scope.

Examples include:

$X \geq$ Threshold A

$X <$ Threshold B

Duration $\geq$ T

LCNRM defines the threshold logic.

LCNSM evaluates the actual state.

Threshold Precision

A threshold should be sufficiently precise for consistent evaluation.

Where an exact numeric threshold is unavailable, the Requirement should identify the authoritative qualitative criterion and preserve its source.

The system should not invent precision.

Therefore:

Undefined Threshold $\neq$ Authority to Manufacture Threshold Value.

Temporal Requirement

A Temporal Requirement defines when or for how long a Condition must apply.

It may specify:

before, after, during, within, for at least, for no longer than, or at a defined lifecycle moment.

Therefore:

Current State $\neq$ Required Historical State by default.

Relevant Time

Every temporal Requirement should identify the relevant time reference where material.

This may be:

occurrence time, action time, Relationship-active period, decision time, Object-state period, or another governed lifecycle moment.

The relevant time must not be silently replaced by the time of later evaluation.

Duration Requirement

Some Conditions require a state to persist for a defined duration.

The Requirement should identify:

the required state, duration, start condition, and end condition where applicable.

Therefore:

Momentary Presence $\neq$ Duration Requirement Satisfaction.

Sequential Requirement

Some Requirements depend on order.

Example:

State A must occur before Action B.

or:

Condition A must be satisfied before Requirement B becomes operative.

The sequence must remain explicit.

Therefore:

Requirement Sequence $\neq$ Requirement Identity.

Concurrent Requirements

Several Requirements may need to be satisfied together.

Canonical structure:

Requirement A AND Requirement B

LCNRM must preserve the conjunction.

Therefore:

Individual Requirement Satisfaction $\neq$ Concurrent Requirement Satisfaction.

Alternative Requirements

A Condition may allow alternative satisfaction paths.

Canonical structure:

Requirement A OR Requirement B

LCNRM must preserve the alternative logic.

Therefore:

Alternative Requirements $\neq$ Concurrent Requirements.

Cumulative Requirements

Several Requirements may contribute cumulatively to a Condition.

LCNRM should preserve:

each component, the accumulation rule, and the point at which the cumulative Requirement becomes evaluable.

Therefore:

Cumulative Requirement $\neq$ Single Requirement Component.

Dependent Requirement

A Requirement may depend on another Condition, Requirement, or governed state.

Canonical structure:

If X applies, Requirement Y must be evaluated.

Dependency must remain explicit.

Therefore:

Requirement Dependency $\neq$ Requirement Satisfaction.

Conditional Requirement

A Requirement may become active only when a defined trigger state exists.

The architecture should distinguish:

trigger, Requirement activation, and Requirement satisfaction.

Therefore:

Trigger Satisfaction $\neq$ Dependent Requirement Satisfaction.

Nested Requirement

A complex Condition may contain Requirements within Requirements.

Where nested logic materially affects evaluation, the structure should remain explicit.

Example:

A AND (B OR C)

must not be flattened into:

A AND B AND C

Therefore:

Nested Requirement Logic $\neq$ Flat Requirement List.

Multiple Requirements for One Condition

One Condition may legitimately contain several Requirement components.

The architecture should preserve whether they are:

concurrent, alternative, sequential, cumulative, or dependent.

Therefore:

Multiple Requirement Components $\neq$ Multiple Conditions by default.

One Requirement Across Multiple Conditions

A common governed criterion may be referenced by several Conditions.

This does not automatically merge the Conditions.

Therefore:

Shared Requirement Criterion $\neq$ Shared Condition Identity.

Requirement Parameters

A Requirement may require explicit parameters such as:

Subject, Object, Relationship, state, action, threshold, time, duration, direction, quantity, or another governed variable.

Parameters should be explicit enough to support State evaluation.

Required Input

LCNRM should identify what type of governed input is necessary to evaluate the Requirement.

Potential inputs may come from:

Operational Reality, evidence, validated records, identity governance, authority governance, Harm governance, system state, or another relevant domain.

Identifying required input does not evaluate that input.

External Governed State as Requirement Input

A Requirement may depend upon a state governed by another OOF® architecture.

For example:

Authority from GOA™, Operational Reality from ORA™, Responsibility from AGA™, Harm from HAGA™, or Identity from OBIDENTITY®.

LCNRM defines how the external state enters the Requirement.

It does not take ownership of the external governance domain.

Requirement Evaluation Criterion

Every Requirement should contain sufficient evaluation criteria to allow LCNSM to determine an explicit Condition State.

The criterion should identify:

what is tested, against what rule, for whom or what, and at what relevant time.

Therefore:

Requirement Description ≠ Evaluation Criterion by default.

Binary Evaluation Criterion

Some Requirements can be evaluated as:

present / absent, occurred / did not occur, met / not met.

Binary evaluation should be used only where the Requirement genuinely supports binary logic.

Unknown or disputed input must not be forced into false binary certainty.

Non-Binary Evaluation Criterion

Some Requirements require states such as:

partial, graded, threshold-based, uncertain, or multi-component.

LCNRM should preserve the appropriate evaluation structure.

Therefore:

Evaluable Requirement ≠ Necessarily Binary Requirement.

Requirement Ambiguity

A Requirement is ambiguous where its meaning, scope, threshold, Subject, Object, Relationship, temporal reference, or evaluation criterion is insufficiently clear.

Ambiguity should be represented explicitly.

Therefore:

Ambiguous Requirement ≠ Evaluable Requirement.

Requirement Uncertainty

Uncertainty may concern:

Requirement meaning, source, scope, applicability parameters, threshold, temporal boundary, or evaluation criterion.

The architecture should preserve the uncertainty rather than invent certainty.

Requirement Conflict

Two applicable sources may appear to produce conflicting Requirements.

LCNRM should preserve:

each Requirement, its source, the nature of the conflict, and unresolved governance state.

Therefore:

Requirement Conflict ≠ Authority to Select a Requirement Arbitrarily.

Competing Requirements

Different Liability Bases may generate competing Requirement structures.

These should remain Basis-specific.

Therefore:

Competing Requirement Structures $\neq$ One Unified Requirement Structure by default.

Requirement Duplication

Several records may express the same underlying Requirement.

LCNRM should distinguish:

duplicate representation, from:

genuinely distinct Requirements.

Therefore:

Multiple Requirement Records $\neq$ Multiple Requirements by default.

Requirement Similarity

Requirements may use similar language while applying to:

different Subjects, different Relationships, different Objects, different Bases, or different times.

Therefore:

Requirement Similarity $\neq$ Requirement Identity.

Requirement Granularity

A Requirement should be decomposed when combining several independent tests would prevent reliable evaluation.

However, decomposition should not create unnecessary fragments.

The governing principle is:

One Requirement should represent one coherent evaluable requirement or one explicitly governed compound requirement.

No Outcome-to-Requirement Shortcut

An adverse outcome must not be used to define retroactively whatever Requirement would make Liability appear established.

Therefore:

Adverse Outcome $\neq$ Condition Requirement.

Requirements derive from the applicable governed Basis and Condition structure.

No Relationship-to-Requirement Shortcut

The existence of a Relationship does not by itself define what the relevant Condition requires.

Therefore:

Liability Relationship $\neq$ Condition Requirement.

The applicable Basis and Condition must establish the Requirement.

No Evidence-to-Requirement Shortcut

Evidence may show what happened.

It does not define what was required.

Therefore:

Evidence $\neq$ Condition Requirement.

The Requirement must exist independently of the evidence used later to evaluate it.

No Claim-to-Requirement Shortcut

A Claim may assert what a Subject believes was required.

The assertion does not establish the governed Requirement.

Therefore:

Claimed Requirement $\neq$ Governed Condition Requirement.

Requirement and Fault

Where Fault is relevant under the applicable Basis, LCNRM may define the Fault-related Requirement.

It must not impose Fault where the Basis does not require it.

Therefore:

Fault Requirement $\neq$ Universal Liability Requirement.

Requirement and Harm

Where Harm is required under an applicable Basis, LCNRM may define the relevant Harm-related Requirement.

Where Harm is not required, LCNRM must not introduce it.

Therefore:

Harm Requirement $\neq$ Universal Liability Requirement.

Requirement and Breach

Where a Condition concerns Breach, the Requirement should identify what underlying obligation, state, action, or nonaction must be evaluated.

Therefore:

Breach Label $\neq$ Complete Condition Requirement.

The Requirement must expose the evaluable structure beneath the label.

Requirement and Knowledge

Where knowledge is conditionally relevant, the Requirement should define:

whose knowledge, knowledge of what, and at what relevant time.

It must not infer historical knowledge from later discovery.

Therefore:

Later Knowledge $\neq$ Required Prior Knowledge.

Requirement and Control

Where control is relevant, the Requirement should define the relevant control characteristic.

Possible distinctions may include:

formal control, operational control, technical control, or another governed form.

Therefore:

Control Label $\neq$ Complete Control Requirement.

Requirement and Authority

Where Authority is relevant, the Requirement should define:

which Authority, held by whom, over what governed space, and at what relevant time.

Authority should not be inferred merely from role title.

Therefore:

Role Label ≠ Established Authority Requirement State.

Requirement and Obligation

Where an obligation forms part of a Condition, the Requirement should distinguish:

existence of obligation, content of obligation, scope of obligation, temporal applicability, and required performance.

Therefore:

Obligation Existence ≠ Obligation Performance.

Requirement and Capability

Where capability matters, the Requirement should define the relevant capability rather than assuming capability from organizational or technical status.

Therefore:

Nominal Capability ≠ Governed Capability Requirement State.

Requirement and Exception Governance

LCNRM may identify that a Requirement is potentially subject to:

an Exception, Exclusion, Waiver, or modifying rule.

It does not determine whether that modifying structure applies.

That belongs to LCNXM.

Therefore:

Potential Requirement Exception ≠ Applicable Exception.

Requirement State Boundary

LCNRM must stop before evaluating whether the Requirement is met.

Its output defines:

what must be evaluated.

LCNSM determines:

what state the Condition is in after evaluation.

This boundary is mandatory.

Requirement Change

A Requirement may later change because:

the applicable Basis changes, the Condition is corrected, its scope changes, a threshold changes, a temporal rule changes, or new authoritative input becomes applicable.

The changed Requirement should remain traceable.

Detailed lifecycle continuity belongs to LCNCM.

Requirement Correction

A Requirement may have been represented incorrectly.

Correction should preserve:

the earlier Requirement, the corrected Requirement, the reason, and the relevant source.

Therefore:

Requirement Correction ≠ Historical Erasure.

Requirement Reassessment

New information may require reassessment of:

Requirement meaning, scope, parameters, threshold, dependency, or evaluation criterion.

Therefore:

Requirement Reassessment $\neq$ Automatic Condition State Change.

LCNSM must separately determine whether the changed Requirement affects the Condition State.

Requirement Version

A materially changed Requirement may require a new version.

The persistent Condition Identity should remain distinguishable from the Requirement version.

Therefore:

New Requirement Version $\neq$ New Liability Condition by default.

Historical Requirement

Later Liability governance may need to determine which Requirement applied at an earlier time.

Therefore:

Current Requirement $\neq$ Historical Requirement by default.

The relevant version and temporal applicability must remain traceable.

Requirement Sufficiency

A Requirement is sufficiently governed when subsequent Condition State governance can determine:

what must be evaluated, which Condition it belongs to, which Basis makes it relevant, which Subject, Relationship, or Object it concerns, what state or action is required, what threshold or temporal rule applies where relevant, what dependencies or alternative paths exist, what evaluation criterion applies, and what material uncertainty remains.

Requirement Sufficiency and State

A sufficiently defined Requirement does not imply that the Condition is satisfied.

Therefore:

Requirement Sufficiency $\neq$ Condition Satisfaction.

It means only that the Condition can now be evaluated coherently.

Handoff to Liability Condition State Governance

Once Requirements are sufficiently governed, LCNRM produces:

Governed Liability Condition Requirement Set

The handoff should preserve:

Condition Identity, Requirement Identity, Requirement Source, Basis reference, Subject reference, Relationship reference, Object reference, Requirement Scope, Required State, action or non-action criteria, thresholds, temporal parameters, dependencies, alternative or concurrent logic, evaluation criteria, uncertainty, conflicts, and applicable Requirement version.

The next governance question becomes:

What is the governed state of each Liability Condition when the relevant inputs are evaluated against its Requirement?

That question belongs to:

LCNSM — Liability Condition State Module.

Minimum Implementation Framework

A conforming implementation of LCNRM must provide at least the following eight capabilities:

Requirement Formation & Identity

Capability to transform Identified Liability Conditions into explicit governed Requirements with stable Requirement identities and traceable sources.

Requirement Scope & Association

Capability to associate Requirements with the correct Liability Basis, Subject, Relationship, Object, and materially relevant scope.

Required State, Action & Absence Governance

Capability to define positive, negative, action, non-action, presence, absence, and other required states without evaluating whether they occurred.

Threshold & Temporal Requirement Governance

Capability to define thresholds, comparison logic, relevant time, duration, ordering, and temporal applicability without manufacturing unavailable precision.

Compound & Dependency Requirement Governance

Capability to represent concurrent, alternative, cumulative, sequential, dependent, conditional, and nested Requirement structures.

Evaluation Criteria & Required Input Governance

Capability to define the evaluation criteria and types of governed input required for subsequent Condition State determination.

Ambiguity, Conflict, Change & Version Governance

Capability to preserve Requirement ambiguity, uncertainty, conflict, duplication, correction, reassessment, change, historical versions, and applicable temporal state.

Requirement Sufficiency & Handoff

Capability to determine whether each Requirement is sufficiently explicit and evaluable for handoff to Liability Condition State governance without determining the Condition State itself.

Governance Logic

The canonical LCNRM governance logic is:

Identified Liability Condition Set

↓

Establish Requirement Identity

↓

Establish Requirement Source

↓

Associate Liability Basis

↓

Associate Subject / Relationship / Object

↓

Define Requirement Scope

↓

Define Required State / Presence / Absence / Action / Non-Action

↓ Define Threshold where applicable ↓ Define Temporal Parameters where applicable ↓

Define Concurrent / Alternative / Sequential / Cumulative Logic

↓

Define Dependencies and Conditional Activation

↓

Define Required Inputs

↓

Define Evaluation Criteria

↓

Preserve Ambiguity / Uncertainty / Conflict

↓

Preserve Applicable Requirement Version

↓

Assess Requirement Sufficiency

↓

Governed Liability Condition Requirement Set

↓

Handoff to Liability Condition State Governance

Governance Outputs

LCNRM may produce:

Liability Condition Requirement

Requirement Identity

Requirement Source Record

Requirement Scope

Basis–Requirement Association

Subject–Requirement Association

Relationship–Requirement Association

Object–Requirement Association

Positive Requirement

Negative Requirement

Action Requirement

Non-Action Requirement

Threshold Requirement

Temporal Requirement

Duration Requirement

Concurrent Requirement Structure

Alternative Requirement Structure

Cumulative Requirement Structure

Sequential Requirement Structure

Dependent Requirement Structure

Conditional Requirement Structure

Nested Requirement Structure

Requirement Parameter Set

Required Input Specification

Requirement Evaluation Criterion

Requirement Ambiguity State

Requirement Uncertainty State

Requirement Conflict State

Requirement Correction Record

Requirement Reassessment Record

Requirement Version Record

Historical Requirement Reference

Requirement Sufficiency State

The principal output is:

Governed Liability Condition Requirement Set

Critical Distinctions

LCNRM preserves the following distinctions:

Condition Requirement ≠ Condition Satisfaction Condition Identity ≠ Condition Requirement Condition Requirement ≠
Condition State Requirement Source ≠ Requirement Satisfaction Requirement Label ≠ Evaluable Requirement
Requirement Scope ≠ Liability Scope Requirement for Subject A ≠ Requirement for Subject B by default Same Subject
≠ Same Requirement across all Relationships Requirement for Object A ≠ Requirement for Object B by default
Requirement under Basis A ≠ Requirement under Basis B by default Negative Requirement ≠ Established Absence
Required Action ≠ Performed Action No Evidence of Action ≠ Established Required Non-Action Undefined Threshold ≠
Authority to Manufacture Threshold Value Current State ≠ Required Historical State by default Momentary Presence ≠
Duration Requirement Satisfaction Individual Requirement Satisfaction ≠ Concurrent Requirement Satisfaction
Alternative Requirements ≠ Concurrent Requirements Cumulative Requirement ≠ Single Requirement Component
Requirement Dependency ≠ Requirement Satisfaction Trigger Satisfaction ≠ Dependent Requirement Satisfaction
Nested Requirement Logic ≠ Flat Requirement List Multiple Requirement Components ≠ Multiple Conditions by default

Shared Requirement Criterion ≠ Shared Condition Identity Requirement Description ≠ Evaluation Criterion by default
Evaluable Requirement ≠ Necessarily Binary Requirement Ambiguous Requirement ≠ Evaluable Requirement
Requirement Conflict ≠ Authority to Select a Requirement Arbitrarily Competing Requirement Structures ≠ One Unified
Requirement Structure by default Multiple Requirement Records ≠ Multiple Requirements by default Requirement
Similarity ≠ Requirement Identity Adverse Outcome ≠ Condition Requirement Liability Relationship ≠ Condition
Requirement Evidence ≠ Condition Requirement Claimed Requirement ≠ Governed Condition Requirement Fault
Requirement ≠ Universal Liability Requirement Harm Requirement ≠ Universal Liability Requirement Breach Label ≠
Complete Condition Requirement Later Knowledge ≠ Required Prior Knowledge Control Label ≠ Complete Control
Requirement Role Label ≠ Established Authority Requirement State Obligation Existence ≠ Obligation Performance
Potential Requirement Exception ≠ Applicable Exception Requirement Correction ≠ Historical Erasure Requirement
Reassessment ≠ Automatic Condition State Change New Requirement Version ≠ New Liability Condition by default
Current Requirement ≠ Historical Requirement by default Requirement Sufficiency ≠ Condition Satisfaction

Cross-Architecture Boundary

LCNRM may define Liability Condition Requirements using governed states originating in other OOF® architectures.

GOA™ may provide Authority, Scope, and Delegation structures referenced by a Requirement.

ORA™ may provide Operational Reality states that later serve as Requirement inputs.

AGA™ may provide Responsibility, Accountability, Authority, and Attribution states.

HAGA™ may provide Harm states where a Liability Condition requires them.

ADIT® may provide evidence used later to evaluate Requirements.

VALIDOS™ may provide validated states or inputs.

OBIDENTITY® may provide Identity, Ownership, Authority, Permission, Provenance, Transfer, and Continuity states.

AIG® and ASGA™ may provide AI or autonomous-system governance states relevant to a Requirement.

The canonical interoperability principle is:

A Liability Condition Requirement may reference an externally governed state without redefining that state or transferring governance ownership of its original domain to LIGA™.

Canonical Boundary

LCNRM begins with an:

Identified Liability Condition Set

and asks:

What exactly must each Condition require so that it can later be evaluated consistently?

LCNRM governs:

Requirement Formation, Requirement Identity, Requirement Source, Requirement Meaning, Requirement Scope, Basis Association, Subject Association, Relationship Association, Object Association, Required State, Required Presence, Required Absence, Required Action, Required Non-Action, Thresholds, Temporal Parameters, Duration, Concurrent Requirements, Alternative Requirements, Cumulative Requirements, Sequential Requirements, Dependencies, Conditional Activation, Nested Requirements, Requirement Parameters, Required Inputs, Evaluation Criteria, Requirement Ambiguity, Requirement Uncertainty, Requirement Conflict, Requirement Correction, Requirement Reassessment, Requirement Versioning, and Requirement Sufficiency.

LCNRM does not determine:

Condition Satisfaction, Condition Failure, final Condition State, Exception Applicability, Exclusion Applicability, Liability-Relevant Connection, Liability Attachment, Liability Determination, Liability Scope, Liability Allocation, Liability State, Transfer, Recourse, Resolution, Discharge, or Finality.

LCNRM ends when a:

Governed Liability Condition Requirement Set

is sufficiently explicit and evaluable for Liability Condition State governance.

Use Case 1 — AI-Enabled Service Requirement

LCNIM has identified a Condition concerning human oversight within an AI-enabled operational service.

LCNRM defines the Requirement by establishing:

which Subject must provide the oversight, which operational activity requires it, the relevant time, the required oversight state, and the criterion against which the Condition can later be evaluated.

LCNRM does not determine whether the oversight actually occurred.

It produces an evaluable Requirement for LCNSM.

Use Case 2 — Industrial Maintenance Requirement

LCNIM has identified a maintenance-related Condition concerning industrial equipment.

LCNRM defines:

which equipment is covered, which Relationship makes the Condition relevant, which Subject the Requirement concerns, what maintenance action or state is required, the applicable period, and any relevant threshold or sequence.

The occurrence of equipment failure does not define or satisfy the Requirement.

LCNSM subsequently evaluates the governed inputs against the Requirement.

Canonical Closing Statement

A Condition cannot be evaluated consistently until what it requires is explicit.

The Requirement must define the relevant Subject, Relationship, Object, state, action, threshold, time, dependency, and evaluation criterion without being rewritten around the outcome that later occurred.

The Requirement defines the test. It does not provide the result.

Define what the Condition requires before determining its state.

Governed Links

Liability Conditions Standard (LCOS)

LIGA™ Architecture Map

LCNSM — Liability Condition State Module

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the MIP® Protection & Licensing terms or contact OOF® directly.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/content/liga/liga-lcos-lcnrm-2.html>

OOF® · MIP® Protected · Free permitted use · Commercial/derivative use may require authorization · Active Protection

Canonical source: <https://originopenfoundation.org/content/liga/liga-lcos-lcnrm-2.html>